Machine Learning Assignment 4

Topics: MLP vs CNN for Image Classification

Total Marks: 10

Note: You can use scikit-learn, Pytorch, and other required libraries.

Dataset: Use the CIFAR-10 dataset available in PyTorch. It consists of 60,000 images of size $32 \times 32 \times 3$ belonging to 10 classes.

Q1. Multi-Layer Perceptron (MLP):

- Load the CIFAR-10 dataset using PyTorch. Normalize the images and visualize a few samples along with their labels. Flatten each image into a vector of size 3072.
- Implement an MLP with at least two hidden layers. Use ReLU activation and softmax output. Train the model using cross-entropy loss.
- Plot training and validation accuracy versus epochs. Plot training loss versus epochs. Report final test accuracy.
- Vary the number of hidden layers and number of neurons. Plot accuracy as a function of model capacity. Comment on overfitting and underfitting behavior.
- Visualize the flattened image as a 1D vector. Randomly shuffle pixels in the input images and retrain the model. Compare performance before and after shuffling.
- Visualize misclassified images.

Q2. Convolutional Neural Network (CNN):

- Implement a CNN using convolutional layers, ReLU activations, and pooling layers (stride as 2). Use fully connected layers at the end for classification. Train the model using cross-entropy loss.
- Plot training and validation accuracy versus epochs. Plot training loss versus epochs. Report final test accuracy. Compare the performance with the MLP model.
- Visualize feature maps from different convolutional layers for a few input images. Observe how features evolve across layers.
- Apply small translations or shifts to input images and evaluate the performance of both MLP and CNN. Compare the robustness of the two models (you can show this experiment in the previous question for MLP).
- Remove pooling layers and observe performance changes.

Boilerplate Code: CIFAR-10 Loading in PyTorch (Google Colab)

```
# ================================
# 1. Install
# ================================
!pip install torch torchvision matplotlib

# ================================
# 2. Imports
# ================================
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# ================================
# 3. Device (GPU if available)
# ================================
```

```python
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# ================================
# 4. Data Transforms
# ================================
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),   # mean
                         (0.5, 0.5, 0.5))   # std
])

# ================================
# 5. Load CIFAR-10 Dataset
# (Automatically downloads)
# ================================
trainset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

testset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

# ================================
# 6. Data Loaders
# ================================
trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True,
    num_workers=2
)

testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=128,
    shuffle=False,
    num_workers=2
)

# ================================
# 7. Class Names
# ================================
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse',
           'ship', 'truck')

# ================================
# 8. Visualize Sample Images
# ================================
def imshow(img):
    img = img / 2 + 0.5   # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.axis('off')

# get random batch
dataiter = iter(trainloader)
images, labels = next(dataiter)

# show images
plt.figure(figsize=(8, 4))
imshow(torchvision.utils.make_grid(images[:8]))
plt.title([classes[labels[j]] for j in range(8)])
plt.show()
```